รายงานคู่มือและผลการทดสอบ

รายวิชา IT Project

ระบบจองคิวร้านตัดผม

ร้านตัดผมโครตซิ่ง

Barbershop Queue Management System

จัดทำโดย

คุณาวุฒิ ดอกบัว	6621600771
ณธนกร ชัยญาติ	6621600836
ณัฐนันท์ จันทร์แจ่ม	6621600844

เสนอ

อาจารย์ สุริยะ พินิจการ

สาขาวิชา เทคโนโลยีสารสนเทศ คณะ ศิลปศาสตร์และวิทยาศาสตร์

มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน

ภาคเรียนที่ 1 ปีการศึกษา 2569

สารบัญ

สารบัญ 1
1 ภาพรวมโครงงาน 3
เป้าหมายของโครงงาน 3
ขอบเขตที่พัฒนา 3
ผลผลิตของโครงงาน 4
2 ฟังก์ชันและสิทธิ์ผู้ใช้งาน 5
ฟังก์ชันหลักของระบบ 5
สถานะของบัตรคิว 6
3 เทคโนโลยีและโครงสร้างระบบ 7
เทคโนโลยีที่ใช้ 7
การเชื่อมต่อหลัก 7
4 ขั้นตอนการทำงานของระบบ 8
ภาพรวมขั้นตอน 8
ขั้นที่ 1 เพิ่มเพื่อนและใช้เมนู LINE 8
ขั้นที่ 2 เลือกบริการและชำระเงิน 9
ขั้นที่ 3 แนบสลิปและรับบัตรคิว 10
ขั้นที่ 4 การแจ้งเตือนทาง LINE 11
ขั้นที่ 5 พนักงานเรียกคิวและให้บริการ 12
5 การพัฒนาและการเชื่อมต่อ API 14
การตรวจสอบข้อมูลและการป้องกัน 15
6 วิธีการทดสอบ 16
ขอบเขตและชุดกรณีทดสอบ 16
สภาพแวดล้อมการรันอัตโนมัติ 16
วิธีรันการทดสอบซ้ำ 17
ความหมายของสถานะ 18

7 ผลการทดสอบ 19
สรุปผล 19
ผลการทดสอบรายกรณี — Unit Test 19
ผลการทดสอบรายกรณี — API End-to-End 20
ผลการทดสอบรายกรณี — UI (Robot Framework) 23
บั๊กที่พบและแก้ไข 24
8 การติดตั้งและการเปิดใช้งาน 26
สิ่งที่ต้องมี 26
รันบนเครื่องพัฒนา 26
ค่าตั้งค่าในไฟล์ .env 26
ตั้งค่า LINE Official Account และ LIFF 27
9 สรุปผลและแนวทางพัฒนาต่อ 28
แนวทางพัฒนาต่อ 28
ภาคผนวก ก คู่มือการใช้งานระบบ 29
ก.1 การเข้าสู่ระบบ 29
ก.2 ลูกค้า: รับบัตรคิว จองล่วงหน้า และดูแต้ม 29
ก.3 พนักงาน: เรียกคิวและให้บริการ 30
ก.4 พนักงาน: ตรวจประวัติการโอน 30
ก.5 ผู้ดูแล: จัดการข้อมูลร้าน 31
ภาคผนวก ข การดูแลระบบเบื้องต้น 33
ข.1 การเปิดและปิดระบบประจำวัน 33
ข.2 การสำรองข้อมูล 33
ข.3 การแก้ปัญหาที่พบบ่อย 33

1 ภาพรวมโครงงาน

ระบบจองคิวร้านตัดผม (ร้านตัดผมโครตซิ่ง) เป็นเว็บแอปสำหรับจัดการคิวลูกค้าของร้านตัดผม ลูกค้ารับบัตรคิวหรือจองล่วงหน้าได้เองจากมือถือผ่าน LINE Official Account (LINE OA) และหน้าเว็บ ชำระค่าบริการด้วย QR PromptPay แล้วแนบสลิป ระบบตรวจสลิปและออกบัตรคิวให้ทันที เมื่อใกล้ถึงคิว ถึงคิว หรือให้บริการเสร็จ ระบบส่งข้อความแจ้งเตือนเข้า LINE ของลูกค้าโดยอัตโนมัติ

ฝั่งร้าน พนักงานใช้หน้าเว็บเรียกคิว ข้ามคิว เริ่มและจบการให้บริการ ดูประวัติการโอนพร้อมรูปสลิป จัดการบริการ ช่าง สมาชิก และดูรายงานยอดขาย หน้าร้านมีจอแสดงคิว (Board) ที่อัปเดตแบบเรียลไทม์ รายงานฉบับนี้อธิบายระบบ เทคโนโลยีที่ใช้ ขั้นตอนการทำงาน ผลการทดสอบ และมีคู่มือการใช้งานพร้อมภาพหน้าจอจริงอยู่ในภาคผนวก ก

เป้าหมายของโครงงาน

- ลดการยืนรอหน้าร้าน ลูกค้ารับคิวจากที่ไหนก็ได้ผ่าน LINE และรู้ล่วงหน้าว่าเหลืออีกกี่คิว
- รับชำระเงินด้วย QR PromptPay และตรวจสลิปอัตโนมัติ ลดงานตรวจยอดโอนด้วยมือ
- แจ้งเตือนลูกค้าเข้า LINE เมื่อใกล้ถึงคิว ถึงคิว ถูกข้าม หรือให้บริการเสร็จ
- ให้พนักงานควบคุมคิวจากหน้าจอเดียว และให้ลูกค้าหน้าร้านเห็นคิวปัจจุบันบนจอ Board
- เก็บข้อมูลยอดขาย การโอน สมาชิก และแต้มสะสม เพื่อใช้ทำรายงาน

ขอบเขตที่พัฒนา

ส่วนงาน	สิ่งที่ทำได้
ลูกค้า (เว็บ / LIFF ใน LINE)	สมัครสมาชิก เข้าสู่ระบบด้วย LINE รับบัตรคิว จองล่วงหน้า ชำระด้วย PromptPay แนบสลิป ดูคิวของฉัน ดูแต้มสะสม
LINE Bot	ตอบคำถามตามคำสำคัญ (คิวของฉัน คิวตอนนี้ ราคา เวลาเปิด ที่ตั้ง แต้ม ฯลฯ) ส่งการ์ดแจ้งเตือนแบบ Flex Message และมี Rich Menu 6 ปุ่ม
พนักงาน / ผู้ดูแล	ภาพรวมประจำวัน จัดการคิว ประวัติการโอนและดูสลิป จองล่วงหน้า สมาชิก ผู้ใช้บริการ รายงาน ตั้งค่าร้าน
จอหน้าร้าน (Board)	แสดงคิวที่ถูกเรียก คิวถัดไป จำนวนคิว และ QR สำหรับเพิ่มเพื่อน LINE โดยไม่ต้องเข้าสู่ระบบ
การทดสอบ	Unit Test, API E2E Test และ UI Automation ด้วย Robot Framework พร้อมรายงานผลเป็นไฟล์ Excel

ผลผลิตของโครงงาน

- ซอร์สโค้ด frontend (React) และ backend (Node.js + Express) พร้อมสคริปต์ฐานข้อมูล PostgreSQL (schema, seed, migrations 001–007)
- LINE OA ที่เชื่อม Webhook, Rich Menu และ LIFF แล้ว
- สคริปต์เปิดระบบ start-system.bat สำหรับเครื่องร้าน
- ชุดทดสอบอัตโนมัติ 97 กรณี และเอกสารผลการทดสอบ เอกสารการทดสอบ_BarberQueue.xlsx
- รายงานคู่มือฉบับนี้

2 ฟังก์ชันและสิทธิ์ผู้ใช้งาน

ระบบแบ่งผู้ใช้เป็น 4 บทบาท (role) เก็บในตาราง users คอลัมน์ role ฝั่ง backend ตรวจสิทธิ์ทุกคำขอด้วย JWT และ middleware authorize() ส่วน frontend ซ่อนเมนูที่ไม่มีสิทธิ์

ตารางที่ 1 บทบาทผู้ใช้และสิทธิ์

บทบาท	หน้าที่	เมนูที่เห็น
ผู้ดูแลระบบ (admin)	ดูแลทั้งระบบ เพิ่ม/แก้ผู้ใช้ทุกบทบาท ตั้งค่าร้าน	ภาพรวม, คิว, ประวัติการโอน, จองล่วงหน้า, สมาชิก, ผู้ใช้, บริการ, รายงาน, ตั้งค่า
พนักงาน (staff)	เรียกคิว รับลูกค้า walk-in ตรวจการโอน จัดการบริการ	เหมือนผู้ดูแล แต่ตั้งผู้ใช้เป็น admin ไม่ได้
ช่าง (barber)	ดูคิวและให้บริการลูกค้า	ภาพรวม, คิว, จองล่วงหน้า, สมาชิก, บริการ, รายงาน
ลูกค้า (customer)	รับคิว จอง ชำระเงิน ดูสถานะคิว	คิวของฉัน, รับบัตรคิว, จองล่วงหน้า, บริการ, โปรไฟล์

ฟังก์ชันหลักของระบบ

ตารางที่ 2 ฟังก์ชันหลัก

ฟังก์ชัน	คำอธิบาย	ผู้ใช้
รับบัตรคิว	เลือกบริการ ⟶ ชำระ PromptPay ⟶ แนบสลิป ⟶ ได้เลขคิว เช่น B-016	ลูกค้า
จองล่วงหน้า	เลือกวัน เวลา บริการ และช่าง ระบบแสดงเฉพาะช่วงเวลาที่ว่าง	ลูกค้า, พนักงาน
เรียกคิว	เรียกคิวถัดไป / เรียกซ้ำ / ข้าม / เริ่มให้บริการ (เลือกช่าง) / เสร็จสิ้น / ยกเลิก	พนักงาน, ช่าง
คิว walk-in	พนักงานออกบัตรคิวให้ลูกค้าที่มาหน้าร้านโดยไม่ต้องมีบัญชี	พนักงาน
ตรวจสลิป	ตรวจยอดเงิน บัญชีผู้รับ และกันสลิปซ้ำด้วย hash ของรูป	ระบบ
ประวัติการโอน	รายการชำระเงินทั้งหมด กรองตามวัน/สถานะ และเปิดดูรูปสลิป	admin, staff
แจ้งเตือน LINE	ใกล้ถึงคิว, ถึงคิว, เรียกซ้ำ, ข้ามคิว, ยกเลิก, เสร็จสิ้น	ระบบ ⟶ ลูกค้า
LINE Bot	ตอบข้อความอัตโนมัติตามคำสำคัญ และ Rich Menu	ลูกค้า
สมาชิกและแต้ม	ดูรายชื่อสมาชิก แต้มคงเหลือ ประวัติแต้ม ผู้ดูแลปรับแต้มได้ ลูกค้าเช็กแต้มผ่าน LINE	ลูกค้า, พนักงาน, ผู้ดูแล
รายงาน	ยอดขายรายวัน จำนวนคิว ผลงานช่าง บริการยอดนิยม	admin, staff, ช่าง

ฟังก์ชัน	คำอธิบาย	ผู้ใช้
จอ Board	จอแสดงคิวหน้าร้าน อัปเดตแบบเรียลไทม์	ทุกคน (ไม่ต้องล็อกอิน)

สถานะของบัตรคิว

บัตรคิวหนึ่งใบ (ตาราง queue_tickets) เปลี่ยนสถานะตามลำดับดังนี้ ทุกครั้งที่เปลี่ยนระบบบันทึกลงตาราง queue_events และกระจายเหตุการณ์ผ่าน SSE ให้ทุกหน้าจออัปเดตทันที

ตารางที่ 3 สถานะของบัตรคิว

สถานะ	ความหมาย	เกิดจาก
waiting	รอเรียก	ชำระเงินสำเร็จ หรือพนักงานออกบัตร walk-in
called	ถูกเรียกแล้ว เชิญที่เคาน์เตอร์	พนักงานกด "เรียกคิวถัดไป" หรือ "เรียกซ้ำ"
serving	กำลังให้บริการ	พนักงานกด "เริ่มให้บริการ" และเลือกช่าง
done	ให้บริการเสร็จสิ้น	พนักงานกด "เสร็จสิ้น" ระบบส่งข้อความขอบคุณทาง LINE
waiting / no_show	ข้ามครั้งแรกเลื่อนไปรอท้ายแถว / ข้ามครบ 2 ครั้งเป็นไม่มาตามคิว	พนักงานกด "ข้าม"
cancelled	ยกเลิก	ลูกค้าหรือพนักงานยกเลิก

3 เทคโนโลยีและโครงสร้างระบบ

ระบบแบ่งเป็น 3 ส่วนหลัก คือ หน้าเว็บ (Frontend) เซิร์ฟเวอร์ API (Backend) และฐานข้อมูล PostgreSQL โดยเชื่อมต่อกับบริการภายนอก คือ LINE Messaging API, LINE Login/LIFF และระบบตรวจสลิป

เทคโนโลยีที่ใช้

ตารางที่ 4 เทคโนโลยีที่ใช้ในระบบ

ส่วน	เทคโนโลยี	หน้าที่
Frontend	React 18, Vite, React Router, Recharts	หน้าเว็บลูกค้า พนักงาน จอแสดงคิว และกราฟรายงาน
Backend	Node.js, Express	REST API ที่ /api ตรวจสิทธิ์ จัดการคิว การชำระเงิน และ LINE
ฐานข้อมูล	PostgreSQL 16	เก็บผู้ใช้ คิว การชำระเงิน สลิป สมาชิก และการตั้งค่า
ยืนยันตัวตน	JWT + Refresh Token, bcrypt	เข้าสู่ระบบ ต่ออายุ session และเข้ารหัสรหัสผ่าน
เรียลไทม์	Server-Sent Events (SSE)	อัปเดตหน้าคิวและจอแสดงคิวทันทีโดยไม่ต้องรีเฟรช
ชำระเงิน	PromptPay QR, EasySlip	สร้าง QR ตามยอด และตรวจสลิปการโอน
LINE	Messaging API, Flex Message, Rich Menu, LINE Login, LIFF	แจ้งเตือน ตอบแชท เมนูลัด และเปิดเว็บใน LINE
เผยแพร่ภายนอก	Cloudflare Tunnel (cloudflared)	ให้ LINE เรียก webhook และเปิด LIFF ผ่าน https
ทดสอบ	node --test, Robot Framework + Browser (Playwright)	Unit Test, API E2E Test และ UI Automation

การเชื่อมต่อหลัก

```
ลูกค้า (LINE / เว็บ) ──https──> Cloudflare Tunnel ──> Frontend :3000 (Vite)
                                                    └─> Backend  :5050 /api
Backend ──SQL──> PostgreSQL (barbershop)
Backend ──push / reply──> LINE Messaging API ──> แชทลูกค้า
LINE ──webhook POST /api/line/webhook──> Backend
Backend ──SSE /api/notifications/stream──> หน้าคิวพนักงาน, จอแสดงคิว (Board)
```

เมื่อมีการเปลี่ยนสถานะคิว Backend จะบันทึกลงฐานข้อมูล ส่งเหตุการณ์ผ่าน SSE ไปยังหน้าจอที่เปิดอยู่ และส่งข้อความแจ้งเตือนทาง LINE ให้ลูกค้าที่ผูกบัญชีไว้

4 ขั้นตอนการทำงานของระบบ

ภาพรวมขั้นตอน

1. ลูกค้าเพิ่มเพื่อน LINE OA ของร้าน หรือสแกน QR หน้าร้าน
2. ลูกค้ากดเมนู "จองคิว & บริการ" เลือกบริการ แล้วสแกน QR PromptPay จ่ายเงิน
3. ลูกค้าแนบสลิป ระบบตรวจสลิปและออกบัตรคิว (เช่น B-016)
4. พนักงานเห็นคิวใหม่ในหน้าคิวทันที กด "เรียกคิวถัดไป" เมื่อพร้อม
5. ระบบส่ง LINE แจ้งลูกค้าเมื่อใกล้ถึงคิว และเมื่อถึงคิว พร้อมแสดงบนจอหน้าร้าน
6. พนักงานกด "เริ่มให้บริการ" แล้ว "เสร็จสิ้น" ระบบส่งข้อความขอบคุณทาง LINE

ขั้นที่ 1 เพิ่มเพื่อนและใช้เมนู LINE

เมื่อเพิ่มเพื่อน ร้านจะส่งข้อความต้อนรับและแสดง Rich Menu 6 ปุ่มด้านล่างแชท ลูกค้าใช้ปุ่มเหล่านี้แทนการพิมพ์



รูปที่ 1 Rich Menu ของร้าน

ตารางที่ 5 ปุ่มใน Rich Menu

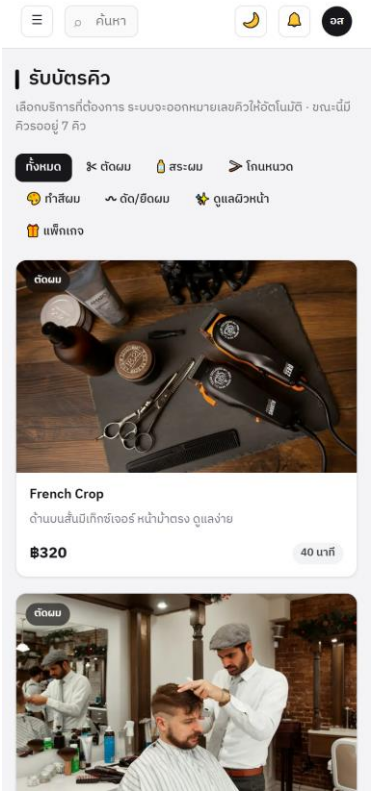
ปุ่ม	การทำงาน
คิวของฉัน	ตอบบัตรคิวปัจจุบัน ลำดับ และเวลารอโดยประมาณ
รายการคิว	ตอบคิวที่กำลังให้บริการและจำนวนคิวที่รอ
ติดต่อร้าน	ตอบเบอร์โทร ที่อยู่ และเวลาเปิดร้าน
จองคิว & บริการ	เปิดหน้ารับบัตรคิวใน LINE (LIFF)
แต้มสะสม	ตอบแต้มคงเหลือของสมาชิก
สมัครสมาชิก	เปิดหน้าโปรไฟล์เพื่อผูกบัญชี LINE กับระบบ



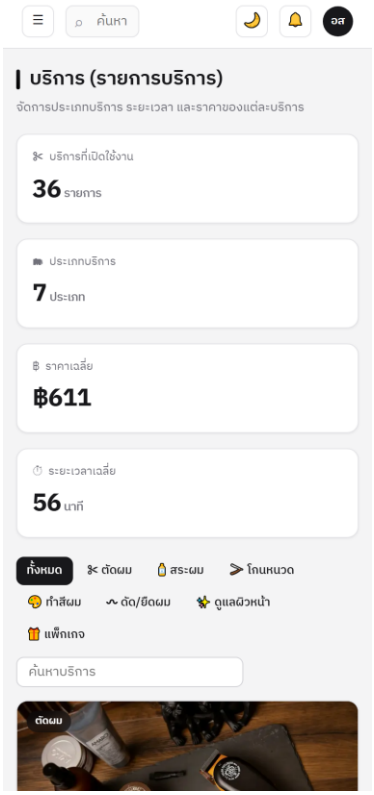
รูปที่ 2 ข้อความต้อนรับใน LINE

ขั้นที่ 2 เลือกบริการและชำระเงิน

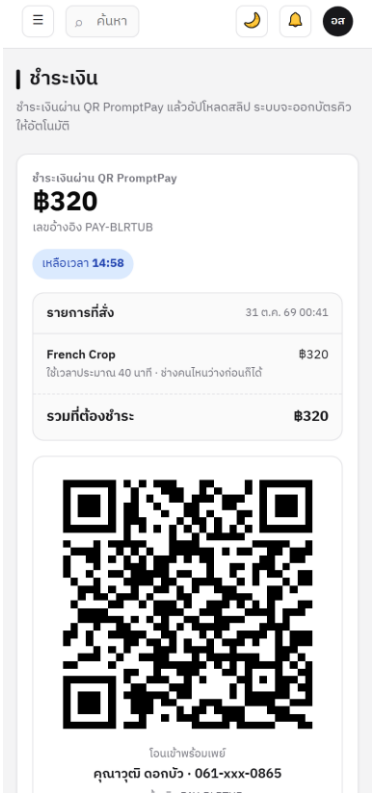
ลูกค้าเลือกบริการ ระบบแสดงราคาและสร้าง QR PromptPay ตามยอดเงิน ลูกค้าสแกนจ่ายด้วยแอปธนาคาร



รูปที่ 3 หน้ารับบัตรคิว



รูปที่ 4 รายการบริการและราคา



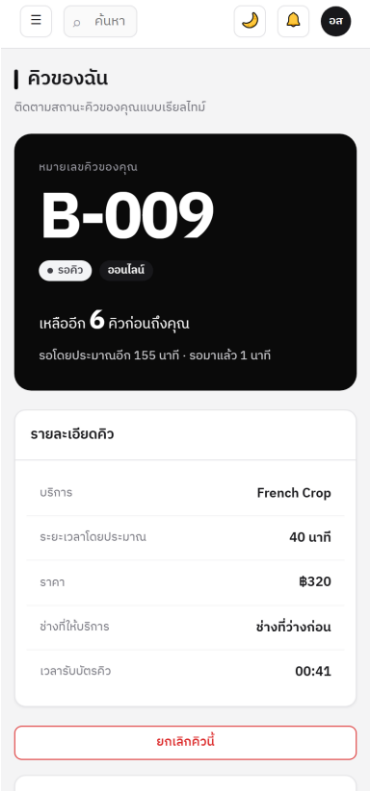
รูปที่ 5 QR PromptPay

ขั้นที่ 3 แนบสลิปและรับบัตรคิว

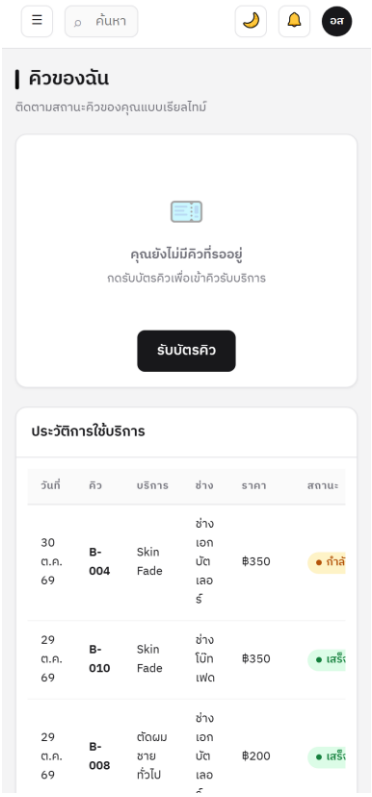
ลูกค้าแนบรูปสลิป ระบบตรวจยอดเงินและกันสลิปซ้ำ เมื่อผ่านจะออกบัตรคิวทันที และหน้า "คิวของฉัน" แสดงลำดับและเวลารอแบบเรียลไทม์



รูปที่ 6 บัตรคิวหลังตรวจสลิป



รูปที่ 7 คิวของฉัน (กำลังรอ)



รูปที่ 8 คิวของฉัน (ถูกเรียก)

ขั้นที่ 4 การแจ้งเตือนทาง LINE

ระบบส่งข้อความ Flex Message เข้าแชทลูกค้าโดยอัตโนมัติ เมื่อเหลืออีก 2 คิว (ตั้งค่าได้) เมื่อถึงคิว และเมื่อให้บริการเสร็จ



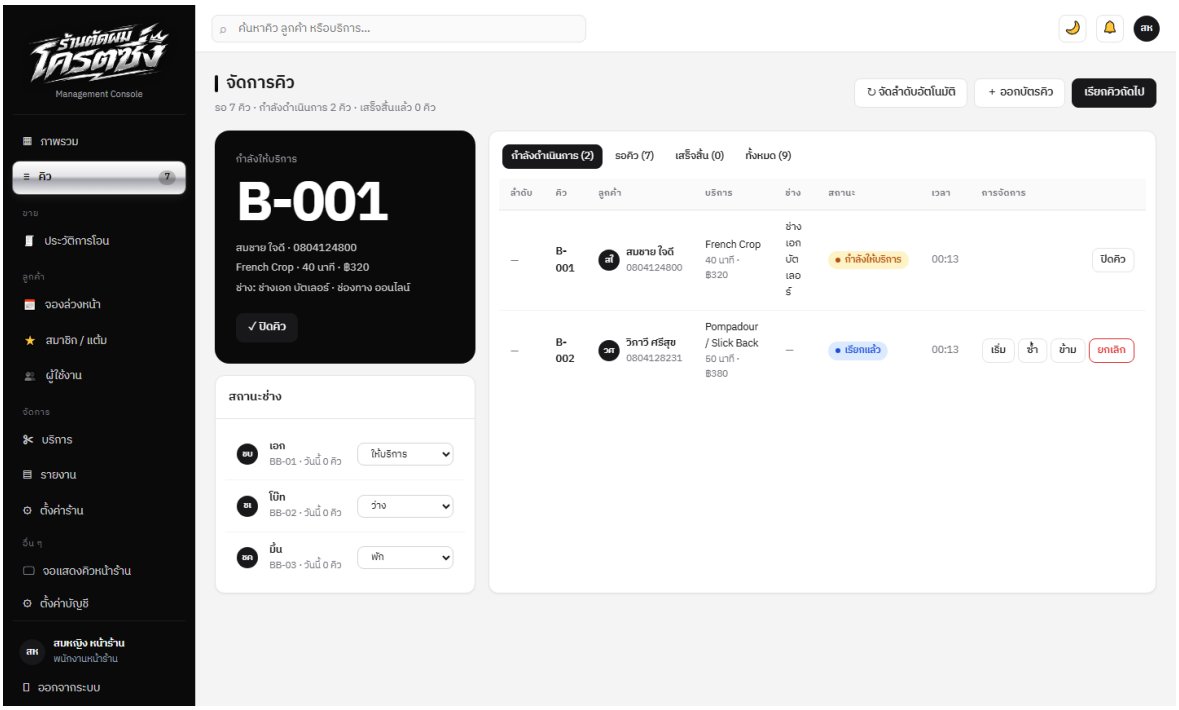
รูปที่ 9 แจ้งถึงคิว B-016



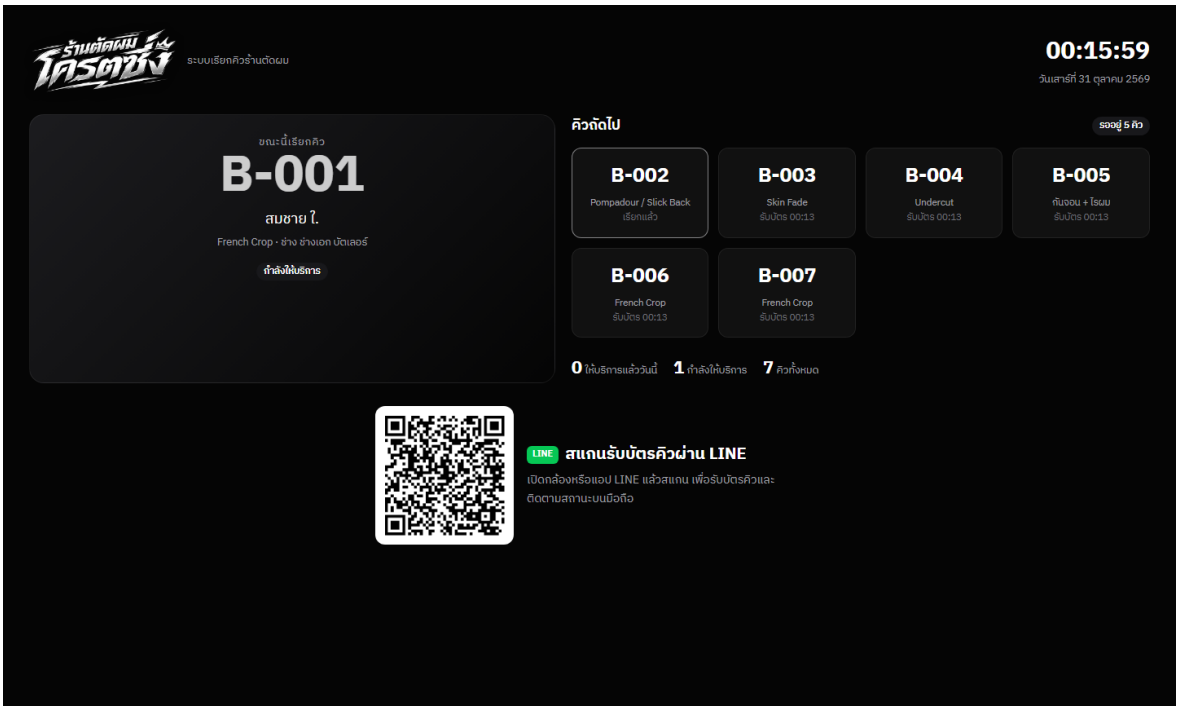
รูปที่ 10 แจ้งให้บริการเสร็จ B-011

ขั้นที่ 5 พนักงานเรียกคิวและให้บริการ

พนักงานเปิดหน้า "คิว" เห็นรายการคิวที่รอ กด "เรียกคิวถัดไป" เลือกช่าง กด "เริ่มให้บริการ" และ "เสร็จสิ้น" ถ้าลูกค้าไม่มา กด "ข้าม" ได้ (ข้ามครั้งแรกคิวเลื่อนไปท้ายแถว ข้ามครบ 2 ครั้งสถานะเป็น "ไม่มาตามคิว")



รูปที่ 11 หน้าจัดการคิวของพนักงาน



รูปที่ 12 จอแสดงคิวหน้าร้าน (Board)

5 การพัฒนาและการเชื่อมต่อ API

Backend เปิด REST API ภายใต้ /api รับส่งข้อมูลเป็น JSON และตรวจสิทธิ์ด้วย JWT ใน header "Authorization: Bearer <token>" ยกเว้นเส้นทางสาธารณะ เช่น จอแสดงคิว การสมัครสมาชิก และ webhook ของ LINE

ตารางที่ 6 กลุ่ม API ของระบบ

กลุ่ม	เส้นทางหลัก	สิทธิ์
ยืนยันตัวตน	POST /auth/register, /auth/login, /auth/refresh, /auth/line, /auth/logout GET /auth/me	สาธารณะ / ผู้ใช้ที่ล็อกอิน
คิว (สาธารณะ)	GET /queue/current, /queue/shop-qr, /queue/track/:token	ทุกคน
คิว (ลูกค้า)	GET /queue/my, POST /queue, POST /queue/:id/cancel	ลูกค้า
คิว (พนักงาน)	GET /queue, POST /queue/call, /queue/:id/call, /recall, /skip, /start, /done, /queue/reorder	พนักงาน, ผู้ดูแล
ชำระเงิน	GET /payments/config, /payments/current POST /payments, /payments/:id/slip, /payments/:id/cancel	ลูกค้า
ประวัติการโอน	GET /payments, /payments/history, /payments/:id/slips POST /payments/:id/approve	พนักงาน, ผู้ดูแล
จองล่วงหน้า	GET /appointments/slots, /appointments/my POST /appointments, /appointments/:id/cancel, /appointments/:id/checkin	ลูกค้า, พนักงาน
บริการ / ช่าง	GET /services, /barbers POST/PUT/DELETE /services, PATCH /barbers/:id/status	ดูได้ทุกคน แก้ไขเฉพาะผู้ดูแล
สมาชิกและแต้ม	GET /loyalty/me, /loyalty, /loyalty/:userId POST /loyalty/:userId/adjust	ลูกค้า (ของตัวเอง), พนักงาน
ผู้ใช้	GET/POST/PUT/DELETE /users, POST /users/:id/reset-password	ผู้ดูแล, พนักงาน
ภาพรวม / รายงาน	GET /dashboard/summary, /dashboard/trend GET /reports/customers, /wait-time, /barbers, /export	พนักงาน, ผู้ดูแล
ตั้งค่า	GET /settings/public, GET/PUT /settings	ผู้ดูแล, พนักงาน

กลุ่ม	เส้นทางหลัก	สิทธิ์
เรียลไทม์	GET /notifications/stream (SSE)	ทุกคน
LINE	POST /line/webhook, GET /line/config, POST /line/link, /line/push	LINE Platform / ผู้ใช้

การตรวจสอบข้อมูลและการป้องกัน

- ตรวจข้อมูลขาเข้าทุกเส้นทางด้วยฟังก์ชันใน backend/src/utils/validate.js เช่น อีเมล เบอร์โทร ความยาวรหัสผ่าน และรหัสบริการ
- รหัสผ่านเก็บแบบ bcrypt ส่วน access token มีอายุสั้นและต่ออายุด้วย refresh token
- คำสั่ง SQL ใช้ parameterized query ทั้งหมด ป้องกัน SQL injection
- การเรียกคิวใช้ SELECT ... FOR UPDATE ในทรานแซกชัน กันพนักงานสองคนเรียกคิวเดียวกันพร้อมกัน
- จำกัดจำนวนครั้งการเข้าสู่ระบบ (20 ครั้ง / 15 นาที) และการสมัครสมาชิก (10 ครั้ง / ชั่วโมง)
- webhook ของ LINE รับ raw body และตรวจลายเซ็น X-Line-Signature ด้วย Channel secret ก่อนประมวลผล
- สลิปถูกตรวจซ้ำด้วยค่า hash ของรูป และยอดเงินในสลิปต้องตรงกับยอดที่ต้องชำระ
- ค่าลับทั้งหมด (รหัสฐานข้อมูล, JWT secret, LINE token, EasySlip key) อยู่ในไฟล์ backend/.env ไม่อยู่ในโค้ด

6 วิธีการทดสอบ

การทดสอบแบ่งเป็น 3 ระดับ ทุกระดับรันอัตโนมัติด้วยคำสั่งเดียว และรันกับฐานข้อมูลทดสอบ barbershop_test ที่แยกจากฐานข้อมูลจริง จึงไม่กระทบข้อมูลลูกค้า

ขอบเขตและชุดกรณีทดสอบ

ตารางที่ 7 ระดับการทดสอบ

ระดับ	เครื่องมือ	สิ่งที่ทดสอบ	จำนวนเคส
Unit Test	node:test	ฟังก์ชันคำนวณฝั่ง backend เช่น QR PromptPay, เวลารอ, แต้ม, การตรวจข้อมูล	38
API End-to-End	node:test + fetch	ยิง REST API จริงตามสถานการณ์ใช้งานของแต่ละบทบาท	46
UI Automation	Robot Framework + Browser (Playwright)	เปิดหน้าเว็บจริงบน Chromium คลิกและตรวจผลบนหน้าจอ	13

สภาพแวดล้อมการรันอัตโนมัติ

ตารางที่ 8 สภาพแวดล้อมการทดสอบ

หัวข้อ	ค่า
ระบบปฏิบัติการ	Windows 10 (10.0.19045)
Node.js	v24.22.1
Python	3.13.16
ฐานข้อมูล	PostgreSQL localhost:5432 · ฐาน barbershop_test
Backend ทดสอบ	http://localhost:5051 · NODE_ENV=test
Frontend ทดสอบ	http://localhost:3001 (vite.test.config.js)
การชำระเงิน	SLIP_PROVIDER=mock · PAYMENT_REQUIRED=false
LINE	ปิด (ไม่ตั้ง token)
เบราว์เซอร์	Chromium (Playwright) headless · 1366×820 · th-TH

ตารางที่ 9 เครื่องมือที่ใช้ทดสอบ

เครื่องมือ	เวอร์ชัน	ประเภท	คำสั่งรัน

เครื่องมือ	เวอร์ชัน	ประเภท	คำสั่งรัน
node:test (Node.js built-in)	v24.22.1	Unit Test	cd backend && npm test
node:test + fetch (API E2E)	v24.22.1	API E2E / Integration	cd backend && npm run test:e2e
Robot Framework	7.5	Automation / Acceptance	automation-testing\.venv\Scripts\robot --outputdir results tests
robotframework-browser (Playwright)	20.6.0	UI Test บนเบราว์เซอร์	(เรียกผ่าน robot)
openpyxl	3.1.5	รายงานผล	python build_report.py
Vite build	-	Build check	cd frontend && npx vite build

วิธีรันการทดสอบซ้ำ

ตารางที่ 10 คำสั่งรันการทดสอบ

ขั้น	คำสั่ง	ผลที่ควรได้
1. สร้างฐานทดสอบ (ครั้งแรก)	cd backend && source tests/test-env.sh && node scripts/setup-db.js	สร้าง barbershop_test + seed + migrations
2. เปิด backend ทดสอบ	cd backend && source tests/test-env.sh && node src/server.js	/api/health ตอบ database=barbershop_test
3. เปิด frontend ทดสอบ	cd frontend && npx vite --config vite.test.config.js	เปิด http://localhost:3001 ได้
4. ติดตั้ง Robot (ครั้งแรก)	cd automation-testing && py -3.13 -m venv .venv && .venv/Scripts/pip install robotframework robotframework-browser openpyxl && .venv/Scripts/rfbrowser init chromium	ติดตั้งครบ
5. รันทุกชุด + สร้างเอกสาร	bash automation-testing/run-all.sh	ได้ไฟล์ เอกสารการทดสอบ_BarberQueue.xlsx
5.1 Unit อย่างเดียว	cd backend && npm test	38 pass
5.2 API E2E อย่างเดียว	cd backend && npm run test:e2e	46 pass
5.3 UI อย่างเดียว	cd automation-testing && .venv/Scripts/robot --outputdir results tests	13 pass · results/report.html

ความหมายของสถานะ

- **PASS** ผลที่ได้ตรงกับผลที่คาดหวังทุกข้อ
- **FAIL** ผลที่ได้ไม่ตรง จะบันทึกเป็นบั๊กในตาราง "บั๊กที่พบและแก้ไข" แล้วรันซ้ำหลังแก้
- **รอบที่เริ่มมี (R1)** รอบทดสอบที่เพิ่มเคสนั้นเข้ามา

7 ผลการทดสอบ

สรุปผล

ตารางที่ 11 สรุปผลการทดสอบ

ระดับ	จำนวนเคส	ผ่าน	ไม่ผ่าน	อัตราผ่าน
Unit Test	38	38	0	100%
API End-to-End	46	46	0	100%
UI (Robot Framework)	13	13	0	100%
รวม	97	97	0	100%

ผลการรันล่าสุดผ่านทั้งหมด 97 จาก 97 เคส ระหว่างทดสอบพบข้อบกพร่อง 22 รายการ ทุกรายการแก้ไขแล้วและมีเคสทดสอบกันถดถอย (ตารางที่แสดงในหัวข้อบั๊กที่พบและแก้ไข)

ผลการทดสอบรายกรณี — Unit Test

ตารางที่ 12 ผล Unit Test

รหัส	ไฟล์	กรณีทดสอบ / ผลที่คาดหวัง	ผล
UT-LY-01	loyalty.test.js	แต้มที่ได้ = ยอด x อัตรา ปัดลง (300 บาท x 0.05 = 15 แต้ม)	PASS
UT-LY-02	loyalty.test.js	ทศนิยมลอยตัวไม่ทำให้แต้มหาย (100 x 0.29 ต้องได้ 29)	PASS
UT-LY-03	loyalty.test.js	ยอด 0 / ติดลบ / อัตรา 0 -> ได้ 0 แต้ม	PASS
UT-LY-04	loyalty.test.js	ส่วนลดเปอร์เซ็นต์ปัด 2 ตำแหน่ง และไม่เกิน 100%	PASS
UT-LY-05	loyalty.test.js	ส่วนลดบาทไม่เกินยอดรวม และค่าติดลบนับเป็น 0	PASS
UT-LY-06	loyalty.test.js	ใช้แต้มได้ไม่เกินยอดคงเหลือ (แต้มละ 1 บาท ยอด 80.5 -> 80 แต้ม)	PASS
UT-LY-07	loyalty.test.js	คืนเงินเต็มจำนวน -> หักแต้มที่ได้ทั้งหมด	PASS
UT-LY-08	loyalty.test.js	คืนบางส่วน -> หักตามสัดส่วน และคืนครั้งที่ 2 ไม่หักซ้ำ	PASS
UT-LY-09	loyalty.test.js	บิลที่ไม่ได้แต้มหรือยอด 0 -> ไม่หักแต้ม	PASS
UT-MS-01	misc.test.js	maskName ปิดนามสกุลเหลืออักษรแรก และชื่อเดี่ยวไม่เปลี่ยน	PASS
UT-MS-02	misc.test.js	settings.num คืน fallback เมื่อค่าไม่ใช่ตัวเลข	PASS
UT-MS-03	misc.test.js	settings.bool รับ "true" และใช้ fallback เมื่อว่าง	PASS
UT-MS-04	misc.test.js	LINE bot จับเจตนาจากข้อความลูกค้า	PASS
UT-MS-05	misc.test.js	LINE bot ข้อความว่างตอบเมนู และข้อความที่ไม่รู้จักเป็น fallback	PASS
UT-MS-06	misc.test.js	รหัสผูกบัญชี LINE จับรูปแบบ LINK-XXXXXX ในข้อความ	PASS

รหัส	ไฟล์	กรณีทดสอบ / ผลที่คาดหวัง	ผล
UT-MS-07	misc.test.js	ApiError มี status code ตามชนิด	PASS
UT-PP-01	promptpay.test.js	crc16 ค่ามาตรฐาน CCITT-FALSE ของ "123456789" = 29B1	PASS
UT-PP-02	promptpay.test.js	crc16 ของสตริงว่าง = FFFF และยาว 4 หลักเสมอ	PASS
UT-PP-03	promptpay.test.js	เบอร์มือถือ 08x แปลงเป็น 0066 + 9 หลัก ใน tag 29/01	PASS
UT-PP-04	promptpay.test.js	มีจำนวนเงิน -> QR ใช้ครั้งเดียว (01=12) และ tag 54 ทศนิยม 2 ตำแหน่ง	PASS
UT-PP-05	promptpay.test.js	ไม่มีจำนวนเงิน / 0 บาท -> QR ใช้ซ้ำ (01=11) และไม่มี tag 54	PASS
UT-PP-06	promptpay.test.js	เลขบัตรประชาชน 13 หลักใช้ tag 02, e-Wallet 15 หลักใช้ tag 03	PASS
UT-PP-07	promptpay.test.js	ตัดขีด/ช่องว่างออกจากเบอร์ก่อนสร้าง QR	PASS
UT-PP-08	promptpay.test.js	CRC ท้าย payload ตรงกับ crc16 ของข้อความก่อนหน้า (รวม "6304")	PASS
UT-PP-09	promptpay.test.js	ไม่ได้ตั้ง PROMPTPAY_ID -> โยน error	PASS
UT-PP-10	promptpay.test.js	maskPromptpay ปิดเลขกลางของเบอร์ และเหลือ 4 ตัวท้ายของเลขยาว	PASS
UT-VD-01	validate.test.js	ช่องบังคับว่าง -> error "กรุณากรอก..."	PASS
UT-VD-02	validate.test.js	ช่องไม่บังคับที่ว่าง ใช้ค่า default	PASS
UT-VD-03	validate.test.js	string ถูก trim และตรวจความยาว min/max	PASS
UT-VD-04	validate.test.js	อีเมลแปลงเป็นตัวพิมพ์เล็ก และปฏิเสธรูปแบบผิด	PASS
UT-VD-05	validate.test.js	เบอร์โทรรับตัวเลข/ขีด และปฏิเสธตัวอักษร	PASS
UT-VD-06	validate.test.js	int ปฏิเสธทศนิยมและค่านอกช่วง min/max	PASS
UT-VD-07	validate.test.js	number ปฏิเสธ Infinity / ตัวอักษร และค่าต่ำกว่า min	PASS
UT-VD-08	validate.test.js	enum รับเฉพาะค่าที่กำหนด	PASS
UT-VD-09	validate.test.js	bool แปลง "true"/"1"/1 เป็น true และค่าอื่นเป็น false	PASS
UT-VD-10	validate.test.js	วันที่ต้องมีจริงในปฏิทิน (2026-02-30 ไม่ผ่าน, 2028-02-29 ผ่าน)	PASS
UT-VD-11	validate.test.js	schema date ตัดเวลาออกเหลือ YYYY-MM-DD	PASS
UT-VD-12	validate.test.js	รหัสผ่านต้องเป็นสตริงอย่างน้อย 6 ตัว	PASS

ผลการทดสอบรายกรณี — API End-to-End

ตารางที่ 13 ผล API End-to-End

รหัส	หมวด	กรณีทดสอบ / ผลที่คาดหวัง	ผล

รหัส	หมวด	กรณีทดสอบ / ผลที่คาดหวัง	ผล
E2E-A-01	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	ดูช่วงเวลาว่างของวันถัดไป (ไม่ต้องล็อกอิน)	PASS
E2E-A-02	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	ลูกค้าจองล่วงหน้า -> 201 และอยู่ในรายการ "การจองของฉัน"	PASS
E2E-A-03	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	จองวันที่ไม่มีจริง (2026-02-30) -> 400	PASS
E2E-A-04	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	จองย้อนหลัง (เมื่อวาน) -> 400	PASS
E2E-A-05	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	จองเกินจำนวนวันล่วงหน้าที่ร้านกำหนด -> 400	PASS
E2E-A-06	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	จำนวนที่จองในช่วงเวลานั้นเพิ่มขึ้นหลังจอง	PASS
E2E-A-07	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	พนักงานเห็นการจองในรายการของวันนั้น	PASS
E2E-A-08	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	ลูกค้าคนอื่นยกเลิกการจองที่ไม่ใช่ของตัวเองไม่ได้	PASS
E2E-A-09	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	ลูกค้ายกเลิกการจองของตัวเอง และยกเลิกซ้ำไม่ได้	PASS
E2E-A-10	B. จองล่วงหน้า ⟶ ยกเลิก / เช็กอินเป็นบัตรคิว	จองวันนี้แล้วพนักงานเช็กอิน -> ได้บัตรคิวแบบมีลำดับความสำคัญ	PASS
E2E-Q-01	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ลูกค้ารับบัตรคิว -> 201 สถานะ waiting มีเลขคิวและลิงก์ติดตาม	PASS
E2E-Q-02	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ลูกค้าเห็นคิวของตัวเองที่ /queue/my พร้อมจำนวนคิวก่อนหน้า	PASS
E2E-Q-03	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ลูกค้ารับบัตรซ้ำขณะยังมีคิวค้าง -> ถูกปฏิเสธ (409)	PASS
E2E-Q-04	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	พนักงานเรียกคิว -> สถานะ called	PASS
E2E-Q-05	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ลูกค้าได้รับแจ้งเตือน "ถึงคิว" หลังถูกเรียก	PASS
E2E-Q-06	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	เริ่มให้บริการ -> สถานะ serving	PASS
E2E-Q-07	A. วงจรคิว: รับคิว ⟶ เรียก	ปิดคิว -> สถานะ done และลูกค้าไม่มีคิวค้าง	PASS

รหัส	หมวด	กรณีทดสอบ / ผลที่คาดหวัง	ผล
	⟶ ให้บริการ ⟶ เสร็จ		
E2E-Q-08	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ปิดคิวที่ปิดไปแล้วซ้ำ -> ถูกปฏิเสธ	PASS
E2E-Q-09	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ข้ามคิวแล้วเรียกใหม่ -> ลูกค้าได้แจ้งเตือนรอบที่ 2	PASS
E2E-Q-10	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	ลูกค้ายกเลิกคิวของตัวเอง -> cancelled	PASS
E2E-Q-11	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	พนักงานออกบัตร walk-in ให้ลูกค้าที่ไม่มีบัญชี	PASS
E2E-Q-12	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	จอคิวสาธารณะ /queue/current ไม่เปิดเผยเบอร์โทรลูกค้า	PASS
E2E-Q-13	A. วงจรคิว: รับคิว ⟶ เรียก ⟶ ให้บริการ ⟶ เสร็จ	id ไม่ใช่ตัวเลข -> 400 (ไม่ใช่ 500)	PASS
E2E-R-01	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	ไม่ได้ล็อกอิน เรียก API ที่ต้องล็อกอิน -> 401	PASS
E2E-R-02	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	token ปลอม -> 401	PASS
E2E-R-03	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	ล็อกอินรหัสผิด -> 401 และรหัสเป็นตัวเลขไม่ทำให้ระบบล่ม	PASS
E2E-R-04	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	ลูกค้าเข้าหน้าจัดการคิว / รายงาน / ผู้ใช้ไม่ได้ -> 403	PASS
E2E-R-05	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	ลูกค้าไม่เห็นอีเมล/เบอร์ของช่างใน /barbers	PASS
E2E-R-06	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	ช่างเรียกคิวได้ แต่จัดการผู้ใช้ไม่ได้	PASS
E2E-R-07	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	พนักงานตั้งตัวเองเป็น admin ไม่ได้ -> 403	PASS
E2E-R-08	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	พนักงานสร้างบัญชี admin ไม่ได้ แต่สร้างบัญชีลูกค้าได้	PASS
E2E-R-09	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	พนักงานแก้ไขบัญชี admin ไม่ได้	PASS
E2E-R-10	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	เฉพาะ admin แก้การตั้งค่าร้านได้	PASS
E2E-R-11	C. สิทธิ์ของแต่ละบทบาท	ลูกค้ากำหนดลำดับความสำคัญคิวเองไม่ได้	PASS

รหัส	หมวด	กรณีทดสอบ / ผลที่คาดหวัง	ผล
	(RBAC) และการยืนยันตัวตน		
E2E-R-12	C. สิทธิ์ของแต่ละบทบาท (RBAC) และการยืนยันตัวตน	ลูกค้ายกเลิกคิวของคนอื่นไม่ได้	PASS
E2E-S-01	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	บันทึกเงินเข้าลิ้นชักมีเศษสตางค์ (150.50) ได้ ไม่ error 500	PASS
E2E-S-02	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	ชำระบิลด้วยเงินสด -> paid คำนวณเงินทอนถูกต้อง	PASS
E2E-S-03	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	จ่ายไม่ครบยอด -> 400	PASS
E2E-S-04	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	สมาชิกได้แต้มสะสมหลังชำระเงิน	PASS
E2E-S-05	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	คืนเงินเต็มบิลสมาชิก -> หักแต้มที่ได้คืน	PASS
E2E-S-06	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	ยกเลิกบิลที่คืนเงินไปแล้ว -> 409 (กันคืนสต็อก/เงินซ้ำ)	PASS
E2E-S-07	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	ใช้แต้มเกินยอดบิล -> 400	PASS
E2E-S-08	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	ลบโค้ดส่วนลดออกจากบิล -> ส่วนลดหายไปด้วย	PASS
E2E-S-09	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	โค้ดที่ใช้ได้ถึงวันนี้ ยังใช้ได้ในวันนี้	PASS
E2E-S-10	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	รายงานส่งวันที่ผิดรูปแบบ -> 400	PASS
E2E-S-11	D. เงินสด / ชำระบิล / แต้มสะสม / คืนเงิน / รายงาน	ส่งออก CSV ใส่ ' หน้าค่าที่ขึ้นต้นด้วย = (กันสูตร Excel)	PASS

ผลการทดสอบรายกรณี — UI (Robot Framework)

ตารางที่ 14 ผล UI Automation

รหัส	กรณีทดสอบ / ผลที่คาดหวัง	ผล
UI-01	หน้าเข้าสู่ระบบแสดงฟอร์มและปุ่มบัญชีทดสอบ	PASS
UI-02	รหัสผ่านผิด แสดงข้อความแจ้งเตือนและยังอยู่หน้าเดิม	PASS
UI-03	พนักงานเข้าสู่ระบบแล้วเห็นเมนูจัดการคิว	PASS
UI-04	ลูกค้าสมัครสมาชิกใหม่ผ่านหน้าเว็บ แล้วเข้าระบบอัตโนมัติ	PASS

รหัส	กรณีทดสอบ / ผลที่คาดหวัง	ผล
UI-05	ลูกค้ารับบัตรคิวผ่านหน้าเว็บ ได้เลขคิวและ QR ติดตาม	PASS
UI-06	ลูกค้าเห็นคิวของตัวเองที่หน้า "คิวของฉัน"	PASS
UI-07	พนักงานเรียกคิวถัดไปจากหน้าจัดการคิว	PASS
UI-08	ลูกค้าเข้าหน้าของพนักงานไม่ได้ (ถูกพากลับ)	PASS
UI-09	หน้าที่ถูกลบ (POS) พากลับหน้าหลัก ไม่ค้างหน้าว่าง	PASS
UI-10	ลูกค้าจองล่วงหน้าผ่านหน้าเว็บ	PASS
UI-11	จอแสดงคิวหน้าร้านเปิดได้โดยไม่ต้องล็อกอิน	PASS
UI-12	เมนู "กะและเงินสด" ถูกลบ และหน้าเดิมพากลับหน้าหลัก	PASS
UI-13	แอดมินเปิดหน้าประวัติการโอน เห็นตารางรายการหรือข้อความว่าไม่มีรายการ	PASS

บั๊กที่พบและแก้ไข

ตารางที่ 15 บั๊กที่พบระหว่างทดสอบ

รหัส	ความรุนแรง	อาการที่พบ	วิธีแก้	สถานะ
BUG-01	Critical	พนักงาน (staff) ตั้งตัวเองหรือสร้างบัญชีใหม่เป็นผู้ดูแลระบบได้ แล้วแก้/ระงับบัญชีแอดมินได้	ให้กำหนด role=admin และแก้บัญชีแอดมินได้เฉพาะแอดมิน · ซ่อนตัวเลือกในหน้าเว็บ	ปิดแล้ว
BUG-02	Major	จอคิวหน้าร้าน (ไม่ต้องล็อกอิน) และสตรีม realtime ส่งเบอร์โทร/รหัสลูกค้าทุกคนออกไป	ส่งเฉพาะข้อมูลสาธารณะ (ตัดเบอร์/รหัสลูกค้า ปิดนามสกุล) · /barbers ไม่ส่งอีเมล/เบอร์ช่างให้ลูกค้า	ปิดแล้ว
BUG-03	Major	รับสลิปโอนเงินที่โอนให้บัญชีอื่นได้ ถ้าสลิปไม่มีเลขพร้อมเพย์ผู้รับ	อ่านผู้รับไม่ได้ต้องเทียบชื่อบัญชีร้าน ไม่ตรง = ไม่รับอัตโนมัติ ให้พนักงานยืนยัน	ปิดแล้ว
BUG-04	Critical	บันทึกเงินเข้า/ออกลิ้นชักที่มีเศษสตางค์ (เช่น 150.50) ขึ้น error 500	ระบุชนิด $3::numeric	ปิดแล้ว
BUG-05	Critical	ยกเลิกบิลที่คืนเงินไปแล้ว ทำให้สต็อกถูกคืนซ้ำและเงินสดที่ควรมีในลิ้นชักผิด	ห้ามยกเลิกบิลที่มีการคืนเงินแล้ว (409)	ปิดแล้ว
BUG-06	Major	คืนเงินแล้วลูกค้ายังได้แต้มสะสม/ยอดใช้จ่าย/ระดับสมาชิกเท่าเดิม	หักแต้มตามสัดส่วนเงินที่คืน (ไม่หักซ้ำ) และลดยอดใช้จ่าย/ปรับระดับสมาชิก	ปิดแล้ว
BUG-07	Major	ลูกค้าเสียแต้มเกินส่วนลดที่ได้จริง เมื่อใส่แต้มมากกว่ายอดบิล	จำกัดแต้มสูงสุดตามยอดคงเหลือ และตรวจซ้ำตอนปิดบิล	ปิดแล้ว
BUG-08	Major	ลบโค้ดส่วนลดออกจากบิลแล้วส่วนลดยังอยู่	ลบโค้ดโดยไม่ได้กรอกส่วนลดมือ = ล้างส่วนลดด้วย	ปิดแล้ว
BUG-09	Major	โค้ดส่วนลด "ใช้ได้ถึงวันที่ X"	เทียบเป็นวันที่ตามเขตเวลาร้าน · ตรวจโค้ดซ้ำตอนปิดบิล (ล็อกแถว	ปิดแล้ว

รหัส	ความรุนแรง	อาการที่พบ	วิธีแก้	สถานะ
		หมดอายุตั้งแต่เที่ยงคืนต้นวัน X (เร็วไป 1 วัน)	กันใช้เกินสิทธิ์)	
BUG-10	Major	ส่งออก CSV แล้วเปิดใน Excel ค่าที่ขึ้นต้นด้วย = + - @ (เช่นชื่อลูกค้า) ทำงานเป็นสูตร	ใส่ ' นำหน้าค่าที่เสี่ยง (CSV injection)	ปิดแล้ว
BUG-11	Major	ส่ง id ที่ไม่ใช่ตัวเลข / วันที่ผิด / limit ติดลบ ได้ error 500 แทน 400	แปลงเป็น 400 · ตรวจวันที่ให้มีจริงในปฏิทิน · จำกัด limit ≥ 1 · Number ต้องไม่ใช่ Infinity	ปิดแล้ว
BUG-12	Major	คิวที่ถูกข้ามแล้วเรียกใหม่ ลูกค้าไม่ได้รับแจ้งเตือนรอบที่ 2	ถ้าเคยแจ้ง your_turn แล้ว ส่ง called_again แทน	ปิดแล้ว
BUG-13	Minor	คิวจองล่วงหน้าที่ถูกข้าม กลับไปอยู่ลำดับที่ 1 ทันที (ข้อความบอกว่าไปท้ายแถว)	เรียง skip_count ก่อน priority (migration 006)	ปิดแล้ว
BUG-14	Minor	สรุปกะนับเงินทอนซ้ำเมื่อบิลจ่ายหลายวิธี · คืนเงินสดไปหักกะที่ขาย ไม่ใช่กะที่คืน	คิดเงินทอนแยก · เพิ่ม sale_refunds.shift_id (migration 006)	ปิดแล้ว
BUG-15	Minor	จองเวลาที่ผ่านไปแล้วของวันนี้ได้ (ผ่าน API) · จองช่างเจาะจงทำให้ช่วงเวลาเกินจำนวนช่าง	ตรวจเวลาตอนจองจริง ใช้เขตเวลาร้าน และนับที่นั่งกติกาเดียว · ยกเลิกแบบมีเงื่อนไขกันชนกับเช็กอิน	ปิดแล้ว
BUG-16	Minor	รหัสผูกบัญชี LINE หนึ่งรหัสถูกใช้ได้ 2 บัญชีถ้าส่งพร้อมกัน	จองรหัสด้วย UPDATE … WHERE used_at IS NULL ก่อน	ปิดแล้ว
BUG-17	Minor	ส่งรหัสผ่าน/เหตุผล/คำค้นเป็นตัวเลขหรือ array แล้วระบบล่ม 500	แปลงเป็น String ก่อนใช้	ปิดแล้ว
BUG-18	Major	ฟอร์มออกบัตรคิว (พนักงาน) ล้างตัวเองระหว่างกรอก/หลังออกบัตร เมื่อมีคิวขยับ	ผูกเฉพาะ open	ปิดแล้ว
BUG-19	Minor	แถบด้านข้าง error เมื่อได้ queue:update ที่ไม่มี tickets · ค้นหาด้านบนไม่ทำงาน · ?src=qr หายหลังล็อกอิน	ตรวจก่อนใช้ / โหลดใหม่ · กรองคิวตาม q · เก็บ pathname + search	ปิดแล้ว
BUG-20	Minor	token หมดอายุแล้วแอปค้างสถานะล็อกอิน ทุกปุ่มขึ้น 401 · SSE ยังใช้ token เก่า	ส่ง event auth:expired ให้ออกจากระบบ · ต่อ SSE ใหม่เมื่อ token เปลี่ยน	ปิดแล้ว
BUG-21	Minor	คิว walk-in ที่ไม่มีชื่อ ได้ชื่อเป็น "" แทน null หลังปิดชื่อ	คงค่า null เดิมไว้	ปิดแล้ว
BUG-22	Minor	ชุดทดสอบอัตโนมัติสมัครสมาชิกถี่ โดน rate limit 429 จนเทสต์ล้ม	ข้าม rate limit เฉพาะ NODE_ENV=test (production ยังจำกัดเท่าเดิม)	ปิดแล้ว

8 การติดตั้งและการเปิดใช้งาน

สิ่งที่ต้องมี

- Node.js 20 ขึ้นไป และ npm
- PostgreSQL 16 (ติดตั้งบนเครื่อง หรือรันผ่าน Docker container ชื่อ barbershop-db)
- บัญชี LINE Official Account และ LINE Developers (สำหรับบอทและ LIFF)
- บัญชีพร้อมเพย์ของร้าน และ API key ของ EasySlip ถ้าต้องการตรวจสลิปจริง

รันบนเครื่องพัฒนา

1. ติดตั้งแพ็กเกจ: รัน **npm install** ในโฟลเดอร์ backend และ frontend
2. คัดลอก backend/.env.example เป็น backend/.env แล้วกรอกค่าฐานข้อมูล, JWT_SECRET และค่า LINE/PromptPay
3. สร้างฐานข้อมูลครั้งแรก: **npm run db:setup** ในโฟลเดอร์ backend (สร้างตาราง + ใส่ข้อมูลตัวอย่าง) เมื่ออัปเดตเวอร์ชันใหม่ใช้ **npm run db:migrate** ซึ่งไม่ลบข้อมูลเดิม
4. ดับเบิลคลิก **start-system.bat** ระบบจะเปิดฐานข้อมูล, backend (พอร์ต 5050), frontend (พอร์ต 3000) และเปิดเบราว์เซอร์ให้
5. เข้าสู่ระบบด้วยบัญชีตัวอย่าง admin@trimco.local รหัสผ่าน 123456 (ควรเปลี่ยนรหัสผ่านก่อนใช้งานจริง)

```
cd backend  && npm install && npm run db:setup
cd frontend && npm install
start-system.bat
```

ค่าตั้งค่าในไฟล์ .env

ตารางที่ 16 ตัวแปรสำคัญใน backend/.env

ตัวแปร	ความหมาย
PGHOST / PGPORT / PGUSER / PGPASSWORD / PGDATABASE	การเชื่อมต่อ PostgreSQL
JWT_SECRET, JWT_EXPIRES_IN, REFRESH_EXPIRES_DAYS	กุญแจและอายุของ token เข้าสู่ระบบ
PUBLIC_URL	URL https ของระบบที่ LINE เรียกเข้ามาได้ (เช่นผ่าน tunnel)
PROMPTPAY_ID, PROMPTPAY_NAME	เบอร์/เลขพร้อมเพย์และชื่อบัญชีร้านที่แสดงบน QR
PAYMENT_REQUIRED, PAYMENT_EXPIRE_MIN	บังคับจ่ายก่อนรับคิว และอายุของ QR (นาที)
SLIP_PROVIDER, EASYSLIP_API_KEY	วิธีตรวจสลิป: mock (ทดสอบ) หรือ easyslip (ตรวจจริง)

ตัวแปร	ความหมาย
LINE_CHANNEL_ACCESS_TOKEN, LINE_CHANNEL_SECRET	กุญแจ Messaging API ของ LINE OA
LINE_BOT_BASIC_ID, LINE_LOGIN_CHANNEL_ID, LIFF_ID	รหัสบัญชีบอท, LINE Login และ LIFF app

ข้อควรระวัง ห้ามนำไฟล์ .env ขึ้น Git หรือส่งให้ผู้อื่น เพราะมี Channel secret, access token และรหัสผ่านฐานข้อมูล

ตั้งค่า LINE Official Account และ LIFF

1. สร้าง LINE Official Account ที่ manager.line.biz แล้วเปิดใช้ Messaging API
2. ปิด "ข้อความตอบกลับอัตโนมัติ" และเปิด Webhook ในหน้าตั้งค่าการตอบกลับ
3. ใน LINE Developers Console ตั้ง Webhook URL เป็น **PUBLIC_URL/api/line/webhook** แล้วกด Verify
4. คัดลอก Channel secret และ Channel access token ไปใส่ใน .env
5. สร้าง channel แบบ LINE Login ใน Provider เดียวกัน แล้วเพิ่ม LIFF app โดยตั้ง Endpoint URL เป็น **PUBLIC_URL/liff**
6. เปลี่ยนสถานะ LINE Login channel จาก Developing เป็น **Published** เพื่อให้ลูกค้าทั่วไปเปิด LIFF ได้
7. สร้างเมนูในห้องแชท: **npm run line:richmenu:image** แล้ว **npm run line:richmenu** ในโฟลเดอร์ backend

9 สรุปผลและแนวทางพัฒนาต่อ

ระบบจองคิวร้านตัดผมทำงานครบตามขอบเขต ลูกค้ารับคิว ชำระเงิน และติดตามคิวได้ทั้งหมดผ่าน LINE ฝั่งร้านเรียกคิว ตรวจประวัติการโอน และดูรายงานได้จากหน้าเว็บ ผลการทดสอบอัตโนมัติผ่าน 97 จาก 97 เคส

แนวทางพัฒนาต่อ

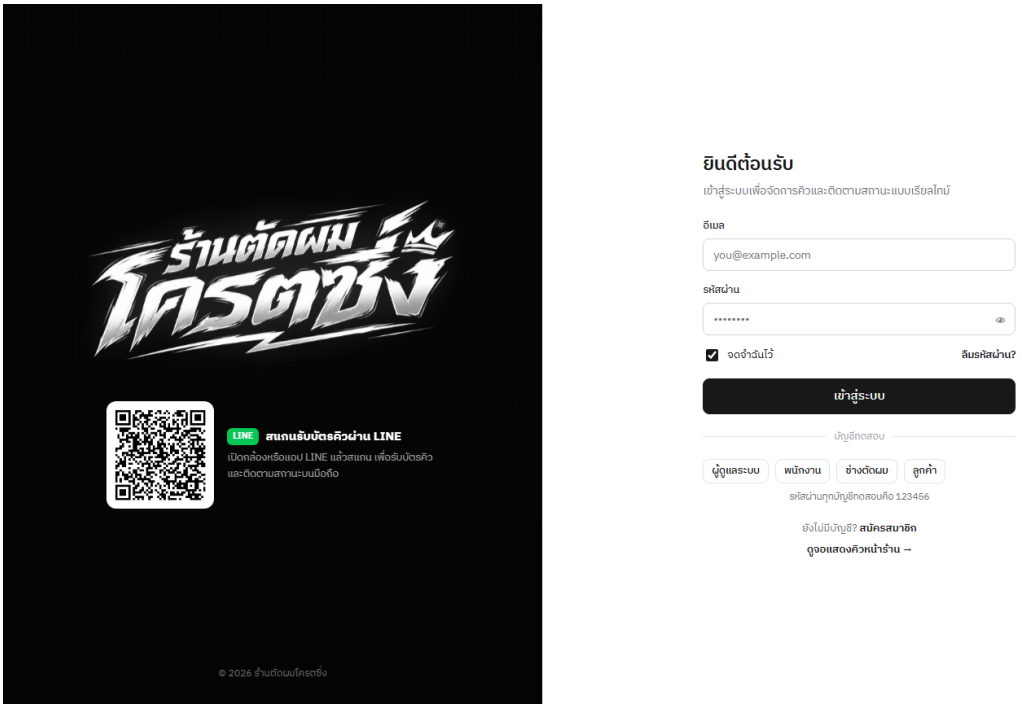
- เปลี่ยนการตรวจสลิปจากโหมดทดสอบ (mock) เป็น EasySlip เพื่อตรวจยอดและบัญชีปลายทางจริง
- ใช้โดเมนถาวรแทน tunnel ชั่วคราว เพื่อไม่ต้องแก้ Webhook URL และ LIFF Endpoint ทุกครั้งที่เปิดระบบ
- เพิ่มการเลือกช่างที่ต้องการตอนรับคิว และแสดงเวลารอแยกตามช่าง
- ส่งสรุปยอดประจำวันให้เจ้าของร้านทาง LINE อัตโนมัติ

ภาคผนวก ก คู่มือการใช้งานระบบ

ภาคผนวกนี้อธิบายการใช้งานทีละขั้นตอนแยกตามบทบาท ลูกค้าใช้งานผ่าน LINE หรือเว็บบนมือถือ พนักงานและผู้ดูแลใช้งานผ่านเว็บบนคอมพิวเตอร์

ก.1 การเข้าสู่ระบบ

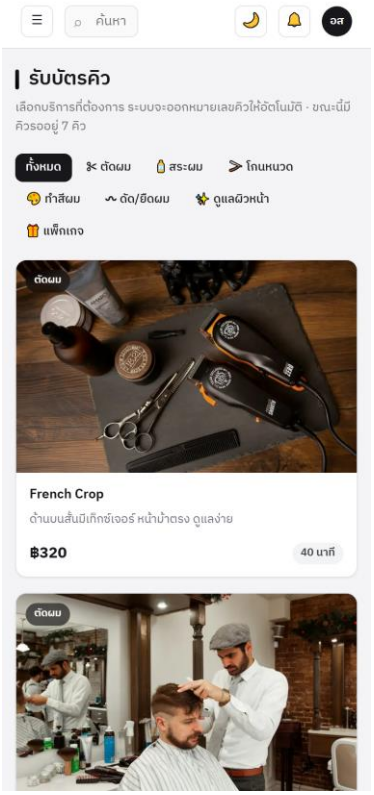
1. เปิดเว็บของร้าน แล้วกด "เข้าสู่ระบบ"
2. กรอกอีเมลและรหัสผ่าน แล้วกดปุ่มเข้าสู่ระบบ
3. ลูกค้าที่เปิดจาก LINE จะเข้าสู่ระบบด้วยบัญชี LINE อัตโนมัติ ไม่ต้องกรอกรหัสผ่าน
4. ระบบพาไปหน้าแรกตามบทบาท: พนักงานไปหน้าภาพรวม ลูกค้าไปหน้าคิวของฉัน



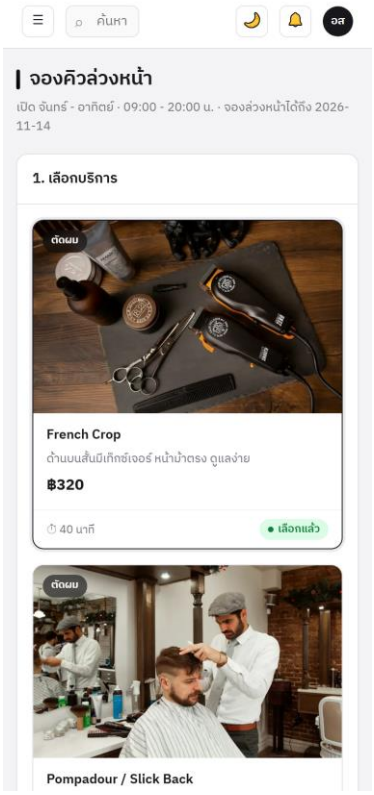
รูปที่ 13 หน้าเข้าสู่ระบบ พร้อม QR สำหรับเพิ่มเพื่อน LINE ของร้าน

ก.2 ลูกค้า: รับบัตรคิว จองล่วงหน้า และดูแต้ม

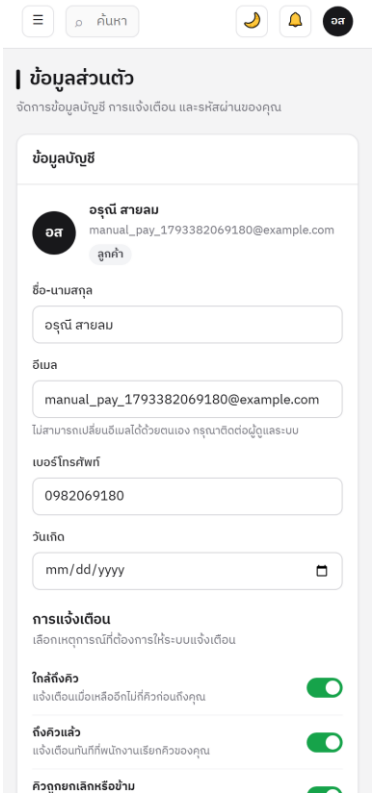
1. เพิ่มเพื่อน LINE ของร้าน (สแกน QR ที่หน้าร้านหรือหน้าเข้าสู่ระบบ)
2. กดเมนู "จองคิว & บริการ" ใน Rich Menu ระบบเปิดหน้ารับบัตรคิวใน LINE
3. เลือกบริการ ชำระเงินด้วย QR PromptPay แล้วอัปโหลดสลิปตามบทที่ 4
4. ถ้าต้องการมาวันอื่น ใช้หน้า "จองล่วงหน้า" เลือกวันและเวลาที่ว่าง
5. กดเมนู "สมัครสมาชิก" เพื่อดูโปรไฟล์และแต้มสะสม



รูปที่ 14 เลือกบริการ



รูปที่ 15 จองล่วงหน้า



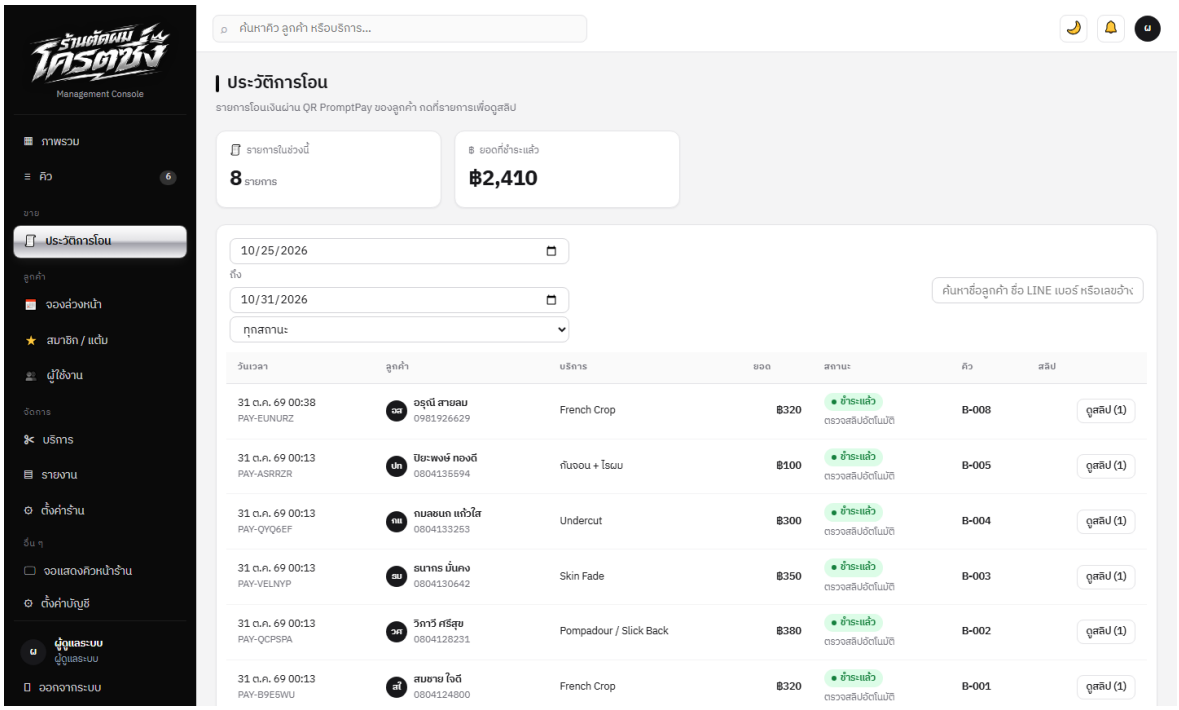
รูปที่ 16 โปรไฟล์และแต้มสะสม

ก.3 พนักงาน: เรียกคิวและให้บริการ

1. เข้าสู่ระบบด้วยบัญชีพนักงาน แล้วเปิดเมนู "คิว"
2. กด "เรียกคิวถัดไป" ระบบส่งข้อความแจ้งลูกค้าทาง LINE และแสดงเลขคิวบนจอหน้าร้าน
3. เมื่อลูกค้ามาถึง เลือกช่างที่ให้บริการ แล้วกด "เริ่มให้บริการ"
4. ตัดเสร็จแล้วกด "เสร็จสิ้น" ระบบส่งข้อความขอบคุณให้ลูกค้า
5. ถ้าลูกค้าไม่มา กด "ข้าม" ครั้งแรกเลื่อนไปท้ายแถว ข้ามครบ 2 ครั้งจะถูกบันทึกว่าไม่มาตามคิว
6. กด "เรียกซ้ำ" เพื่อแจ้งเตือนลูกค้าที่ถูกเรียกแล้วอีกครั้ง

ก.4 พนักงาน: ตรวจประวัติการโอน

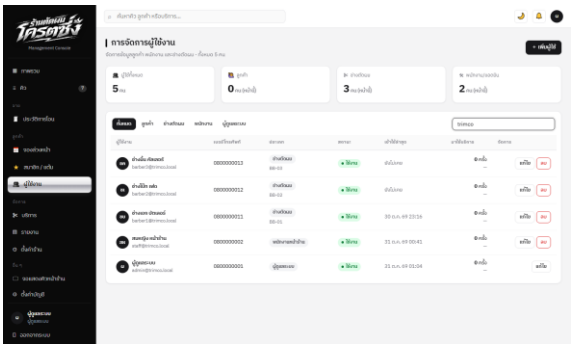
1. เปิดเมนู "ประวัติการโอน" เพื่อดูรายการชำระเงินทั้งหมด พร้อมยอด เวลา และสถานะ
2. กดที่รายการเพื่อเปิดรูปสลิปที่ลูกค้าอัปโหลด และกด "ดาวน์โหลดรูปสลิป" ถ้าต้องการเก็บไฟล์
3. ค้นหาด้วยชื่อลูกค้า ชื่อ LINE เบอร์ หรือเลขอ้างอิง และกรองตามวันที่ได้



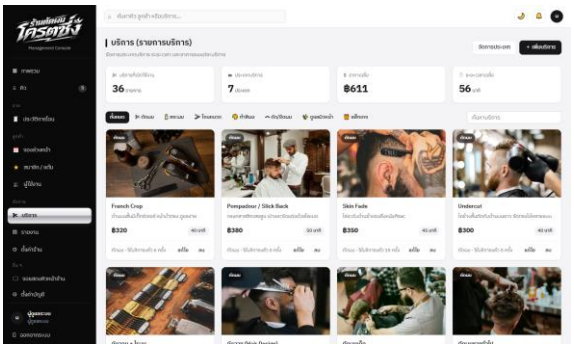
รูปที่ 17 หน้าประวัติการโอน

ก.5 ผู้ดูแล: จัดการข้อมูลร้าน

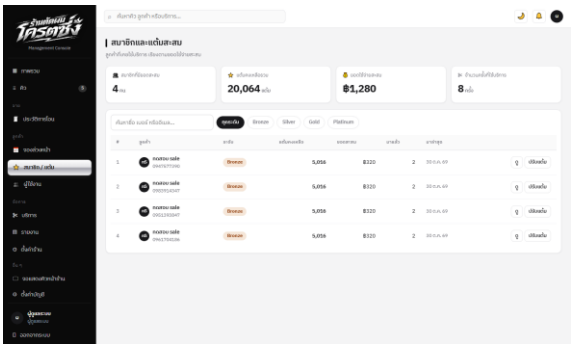
- **ผู้ใช้**: เพิ่มพนักงาน กำหนดบทบาท ปิดบัญชี และตั้งรหัสผ่านใหม่ แสดงชื่อและรูปจาก LINE ของลูกค้า
- **บริการ**: เพิ่มหรือแก้ไขชื่อบริการ ราคา และเวลาที่ใช้
- **สมาชิก**: ดูแต้มคงเหลือและประวัติแต้ม ปรับแต้มเองได้
- **รายงาน**: ดูยอดรายได้ จำนวนคิว และบริการยอดนิยมตามช่วงวันที่
- **ตั้งค่า**: ข้อมูลร้าน เวลาเปิดปิด ตัวอักษรนำหน้าเลขคิว จำนวนคิวก่อนแจ้งเตือน อัตราแต้มสะสม และโค้ดส่วนลด



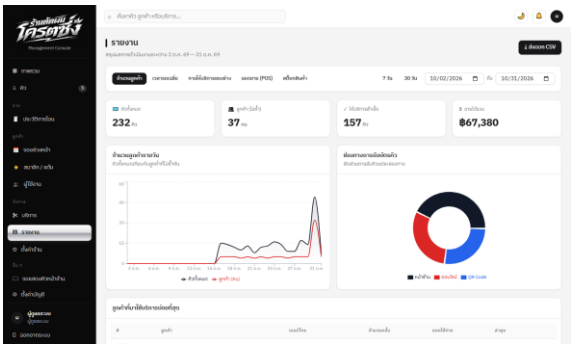
รูปที่ 18 หน้าผู้ใช้



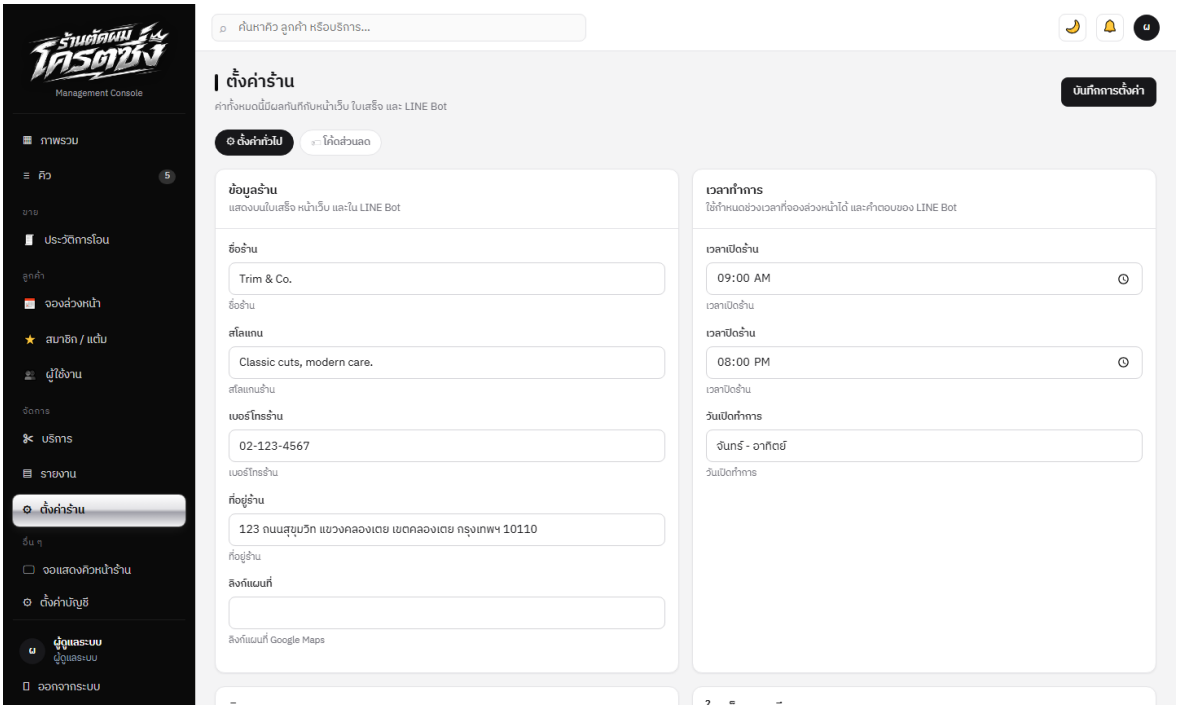
รูปที่ 19 หน้าบริการ



รูปที่ 20 หน้าสมาชิก



รูปที่ 21 หน้ารายงาน



รูปที่ 22 หน้าตั้งค่าร้าน

ภาคผนวก ข การดูแลระบบเบื้องต้น

ข.1 การเปิดและปิดระบบประจำวัน

1. ดับเบิลคลิก **start-system.bat** ระบบจะเปิดฐานข้อมูล อัปเดตตาราง แล้วเปิด Backend และ Frontend
2. เปิด http://localhost:3000 เพื่อใช้งาน และเปิด /board บนจอหน้าร้าน
3. ปิดระบบโดยปิดหน้าต่าง Backend และ Frontend ที่สคริปต์เปิดไว้

ข.2 การสำรองข้อมูล

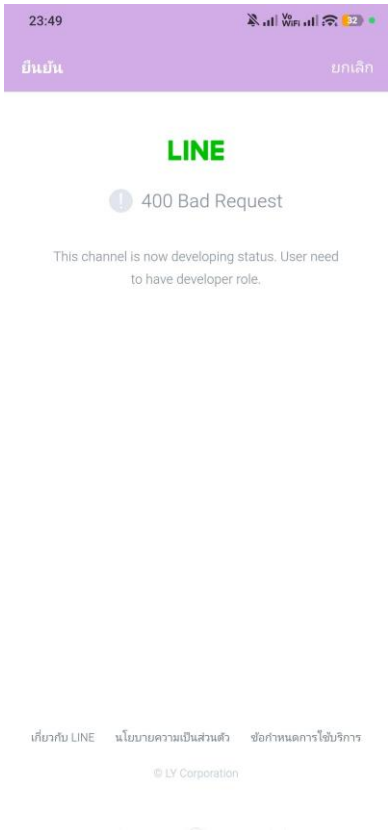
สำรองฐานข้อมูลก่อนอัปเดตระบบหรือรัน migration ทุกครั้ง

```
pg_dump -U postgres -d barbershop -F c -f backup_YYYYMMDD.dump
pg_restore -U postgres -d barbershop --clean backup_YYYYMMDD.dump
```

ข.3 การแก้ปัญหาที่พบบ่อย

ตารางที่ 17 ปัญหาที่พบบ่อยและวิธีแก้

อาการ	สาเหตุ	วิธีแก้
LINE แสดง 400 Bad Request เมื่อเปิด LIFF	LIFF app ยังอยู่สถานะ Developing	ใน LINE Developers กด Publish ช่อง LINE Login
ลูกค้าไม่ได้รับข้อความแจ้งเตือน	ยังไม่ได้ผูกบัญชี LINE หรือ Webhook URL ผิด	ให้ลูกค้าเปิดผ่านริชเมนู และตรวจ Webhook URL ใน LINE Developers
Webhook ใช้ไม่ได้หลังรีสตาร์ต	URL ของ tunnel ชั่วคราวเปลี่ยน	อัปเดต PUBLIC_URL ใน .env และ Webhook/Endpoint URL ใน LINE
ตรวจสลิปไม่ผ่าน	ยอดเงินไม่ตรง สลิปซ้ำ หรือรูปไม่ชัด	ให้ลูกค้าอัปโหลดใหม่ หรือตรวจในหน้าประวัติการโอน
หน้าเว็บโหลดข้อมูลไม่ได้	Backend หรือฐานข้อมูลไม่ทำงาน	เปิด http://localhost:5050/api/health และรัน start-system.bat ใหม่



รูปที่ 23 ข้อผิดพลาด 400 เมื่อ LIFF ยังไม่เผยแพร่